Neural Networks 206 (2027) 109863

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

An adaptive gradient compensation neural network for time-varying nonlinear optimization with application to redundant manipulators

Shangfeng Du [a,b,c], Duo Liu [a,b,c,*], Yang Si [d], Haoen Huang [e], Zihao Zhang [a,b,c]

[a] School of Big Data and Software Engineering, Chongqing University, Chongqing, 401331, China
[b] Jialingjiang Laboratory, Chongqing, 401331, China
[c] Jialingjiang Laboratory, Chongqing University, Chongqing, 401331, China
[d] College of Electrical Engineering and Automation, Shandong University of Science and Technology, Qingdao, 266590, China
[e] School of Artificial Intelligence and Automation and the Key Laboratory of Image Processing and Intelligent Control of Education Ministry of China, Huazhong University of Science and Technology, Wuhan, China

ARTICLE INFO

Keywords:
Adaptive gradient compensation neural network
Time-varying nonlinear optimization problems
Redundant manipulators
Trajectory tracking

ABSTRACT

Gradient neural networks (GNNs) provide an effective approach for solving time-varying nonlinear optimization (TVNO) problems but suffer from inherent lagging errors. Existing GNN acceleration methods generally strengthen feedback correction without explicitly compensating for the drift induced by time variation and often face a trade-off between convergence speed and numerical stability. To address these limitations, this paper proposes an adaptive gradient compensation neural network (AGCNN). It integrates a feedforward velocity compensation mechanism to explicitly compensate for the drift induced by time variation and an adaptive spectral compensation mechanism to enhance feedback correction under weak-curvature conditions; together, the two mechanisms eliminate the inherent lagging error of GNNs. An enhanced power-exponential magnitude activation function (EPMAF) is further developed to improve convergence performance and robustness to noise. Theoretical analysis establishes global finite-time convergence and robustness under bounded disturbances, with explicit bounds on convergence time and steady-state error. Numerical results show that AGCNN achieves convergence times on the order of 10^{-3}–10^{-2} s and maintains the maximum steady-state residual error on the order of 10^{-4}–10^{-3} under different noise conditions. Experiments on a Franka Emika Panda manipulator further validate its effectiveness, with maximum position errors on the order of 10^{-4} m.

1. Introduction

Time-varying nonlinear optimization (TVNO) problems have long been a fundamental topic in automation systems and engineering, and they arise in a wide range of applications, including signal processing (Luo et al., 2025), machine learning (Chen et al., 2024a; Huang et al., 2025), and robotics (Fu et al., 2023; Liufu et al., 2026). Classical numerical methods, such as the Newton-Raphson method (Fu et al., 2022) and gradient descent (Andrychowicz et al., 2016), as well as heuristic strategies including particle swarm optimization (Zhao et al., 2023), genetic algorithms (Datta et al., 2016), and evolutionary algorithms (Liao et al., 2020), have demonstrated effectiveness for static optimization problems. However, most of these methods follow a discrete-time serial computing paradigm. When the problem data vary continuously over time, this stop-and-go computational mode inevitably incurs response delays, thereby limiting its suitability for dynamic environments.

In time-critical tasks, such as real-time robot trajectory tracking (Yan et al., 2025) and dynamic obstacle avoidance (Xie et al., 2025), computation-induced delay may directly degrade decision quality and tracking accuracy. Although faster hardware can reduce iteration time, it cannot fundamentally remove the delay caused by serial iterative computation. Therefore, developing computational schemes with fast dynamic response and low tracking error remains a critical challenge for TVNO problems.

To overcome the latency limitations of iterative optimization algorithms, various approaches have been explored for dynamic control and optimization: model predictive control (MPC) (Tang et al., 2025) and reinforcement learning (RL) (Lyu et al., 2025) have been widely applied to complex dynamic control and decision-making tasks, with MPC typically relying on repeated online optimization and RL on extensive training to learn effective control policies. In this context, recurrent neural networks (RNNs) have emerged as an effective alternative for dynamic opti-

* Corresponding author at: School of Big Data and Software Engineering, Chongqing University, Chongqing, 401331, China.
E-mail addresses: dushangfeng@stu.cqu.edu.cn (S. Du), liuduo@cqu.edu.cn (D. Liu), 202481080007@sdust.edu.cn (Y. Si), henhuang@hust.edu.cn (H. Huang), 20232401002@stu.cqu.edu.cn (Z. Zhang).

https://doi.org/10.1016/j.neunet.2026.109863
Received 20 April 2026; Received in revised form 12 August 2026; Accepted 1 November 2026
Available online 5 November 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

mization (Zhang et al., 2024; Zhang & Ge, 2005; Zhang et al., 2002). By exploiting their inherent parallel processing capability, RNNs can generate real-time solutions efficiently in time-varying environments (Guo & Zhang, 2012; Jin et al., 2023b; Li et al., 2024; Tan et al., 2024; Xiao et al., 2024), and have further been applied to matrix decomposition problems under noise interference and data loss (Yang et al., 2026, 2025a,b,c). As a specialized class of RNNs, gradient neural networks (GNNs) (Liao et al., 2022a; Zhang et al., 2023) drive the error to zero along the negative gradient direction, making them particularly suitable for tracking time-varying optimal solutions. Nevertheless, GNNs suffer from a critical structural deficiency: the parameter drift of the time-varying objective function inevitably restricts the dynamical system to a bounded lagging error (Zhang et al., 2009). In essence, this lagging behavior is jointly influenced by the drift intensity induced by temporal variation and the local curvature of the objective function. A stronger drift or a weaker curvature generally leads to a larger tracking error. This observation suggests that reducing lagging error requires not only faster error correction but also a mechanism that explicitly addresses the drift source while maintaining sufficient correction capability under heterogeneous curvature conditions.

To mitigate this inherent lag and improve real-time performance, recent studies have mainly pursued two acceleration strategies: (1) designing advanced activation functions (AFs), and (2) introducing time-varying coefficients. Regarding the design of AFs, researchers have progressed from conventional monotone odd mappings to nonlinear and even nonconvex forms in order to reduce convergence time and enhance robustness. Jin et al. presented a power piecewise AF that achieves zero error within fixed time (Jin et al., 2023a). Kong et al. developed robust nonlinear AFs to guarantee predefined convergence in the presence of noise (Kong et al., 2023). Jiang et al. further proposed a nonconvex saturation AF with anti-perturbation capability (Jiang et al., 2021). For the coefficients, Liao et al. introduced an adaptive gain strategy to accelerate the convergence of GNNs in solving dynamic linear matrix equations (Liao et al., 2022b). Inspired by fuzzy control, Dai et al. designed a fuzzy adaptive strategy to accelerate neural network convergence (Dai et al., 2022).

Although these efforts have achieved encouraging results, several issues remain insufficiently addressed. First, when these acceleration strategies are applied within GNNs, they act indirectly through nonlinear mappings or adaptive gains and leave unaddressed the drift term induced by variations in the objective function over time. As a result, the lagging error may be reduced, whereas its structural source remains unresolved. Second, when the objective function becomes weakly curved, the effectiveness of feedback correction may deteriorate significantly. In such cases, even drift compensation can introduce an undesirable tension between tracking improvement and robustness, because the correction mechanism may become more sensitive to perturbations. Third, many AFs still involve a trade-off between transient convergence speed and local numerical stability, especially in the presence of noise or near equilibrium. These limitations collectively raise a fundamental question: how can one explicitly compensate for time-varying drift while preserving effective and robust feedback correction under heterogeneous curvature and achieving fast convergence?

To address the above limitations, this paper proposes an adaptive gradient compensation neural network (AGCNN) for solving TVNO problems. First, a feedforward velocity compensation mechanism is introduced to explicitly compensate for the drift induced by time variation. On this basis, an adaptive spectral compensation mechanism is constructed to enhance feedback correction under weak-curvature conditions. Together, the two mechanisms eliminate the inherent lagging error of GNNs. Finally, an enhanced power-exponential magnitude activation function (EPMAF) is developed to improve convergence performance and robustness to noise. By strengthening the nonlinear correction effect for large errors, moderating output growth, and preserving sufficient sensitivity near equilibrium, EPMAF helps accelerate convergence while alleviating noise sensitivity. Comprehensive theoretical analysis, numerical simulations, and trajectory tracking experiments on a Franka Emika Panda manipulator are conducted to verify the effectiveness of AGCNN. The main contributions of this paper are summarized as follows:

- The feedforward velocity compensation mechanism and adaptive spectral compensation mechanism are proposed to eliminate the inherent lagging error of GNNs
- An EPMAF is developed to accelerate convergence and alleviate stiffness and noise sensitivity near the equilibrium.
- The AGCNN is theoretically analyzed for convergence and robustness, and validated through numerical simulations and physical trajectory tracking experiments on a Franka Emika Panda manipulator.

2. Problem formulation and preliminaries

2.1. TVNO problem

Consider the following TVNO problem:

$$\min_{\boldsymbol{x}(t)\in\mathbb{R}^n} \mathcal{Q}(\boldsymbol{x}(t),t), \quad \forall t\in[0,\infty), \tag{1}$$

where $\boldsymbol{x}(t)\in\mathbb{R}^n$ is the decision variable vector, $t\in[0,\infty)$ denotes time, and $\mathcal{Q}(\boldsymbol{x},t)$ is a smooth and strictly convex nonlinear objective function. In addition, the Hessian matrix $\nabla^2_{\boldsymbol{x}}\mathcal{Q}(\boldsymbol{x},t)$ is assumed to be positive definite, with its smallest eigenvalue bounded below by a positive constant $\underline{\lambda}$ independent of t, which guarantees the existence and uniqueness of the optimal solution.

By the first-order optimality condition, the optimal trajectory $\boldsymbol{x}^*(t)$ satisfies

$$\nabla_{\boldsymbol{x}}\mathcal{Q}(\boldsymbol{x}^*(t),t)=\boldsymbol{0}.$$

For notational convenience, define

$$\boldsymbol{g}(\boldsymbol{x}(t),t)\triangleq\nabla_{\boldsymbol{x}}\mathcal{Q}(\boldsymbol{x}(t),t).$$

2.2. GNN

An RNN can be formulated as a non-autonomous dynamical system with n interconnected neurons:

$$\dot{x}_i(t)=\mathcal{R}_i(x_1,\dots,x_n,t), \quad i=1,\dots,n, \tag{2}$$

where $\mathcal{R}_i(\cdot)$ denotes the network mapping associated with the ith state variable $x_i(t)$. To quantify the discrepancy between the current state and the theoretical optimum, we define the error variable of the ith neuron as

$$\varepsilon_i(t)=g_i(\boldsymbol{x}(t),t)=\frac{\partial\mathcal{Q}}{\partial x_i}. \tag{3}$$

As a specific class of RNNs, a GNN evolves along the negative gradient of the energy function $L(t)=\frac{1}{2}\sum_{i=1}^n\varepsilon_i^2(t)$, thereby avoiding the computational burden of matrix inversion.

The dynamics of the ith neuron in the dynamical system are given by

$$\dot{x}_i(t)=-\gamma\frac{\partial L}{\partial x_i}=-\gamma\sum_{j=1}^n\mathcal{D}_{ji}(\boldsymbol{x}(t),t)\varepsilon_j(t), \tag{4}$$

where $D_{ji}=\partial^2\mathcal{Q}/\partial x_j\partial x_i$ denotes the synaptic coupling strength from the jth neuron to the ith neuron.

Note that the time-varying coefficient matrix $D(\boldsymbol{x}(t),t)$ characterizes the local curvature of the objective function $\mathcal{Q}(\boldsymbol{x}(t),t)$ (Goldfarb, 1980). In GNN (4), $D(\boldsymbol{x}(t),t)$ shapes the convergence dynamics of the error by scaling and reorienting the error vector. Hence, drastic fluctuations in $D(\boldsymbol{x}(t),t)$ may compromise the stability and convergence of the dynamical system. For a rigorous analysis, the spectral properties are investigated through eigenvalue decomposition. Define the time-varying eigenvalue set Λ_t as

$$\Lambda_t=\{\lambda_j(t)\mid\lambda_j(t)\in\mathbb{R}_+,\ j=1,\dots,n\},$$

where $\lambda_j(t)$ denotes the jth eigenvalue of $\mathcal{D}(\boldsymbol{x}(t),t)$. Then, the eigenvalues satisfy

$$0 < \underline{\lambda} \le \lambda_{\min}(t) \le \lambda_j(t) \le \lambda_{\max}(t),$$

where $\lambda_{\min}(t) = \min_j\{\lambda_j(t)\}$ and $\lambda_{\max}(t)$ represents the spectral norm (or maximum singular value) of the matrix $\mathcal{D}(\boldsymbol{x}(t),t)$, i.e., $\lambda_{\max}(t) = \|\mathcal{D}(\boldsymbol{x}(t),t)\|_2 = \max_j\{\sqrt{\lambda_j^2(t)}\}$.

According to matrix diagonalization theory, the symmetric product $\mathcal{D}(t)\mathcal{D}^{\mathrm{T}}(t)$ is unitarily equivalent to a diagonal matrix representation that evolves over time, satisfying:

$$\mathcal{D}(t)\mathcal{D}^{\mathrm{T}}(t) \cong \begin{bmatrix} \lambda_1^2(t) & 0 & \cdots & 0 \\ 0 & \lambda_2^2(t) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^2(t) \end{bmatrix}.$$

This decomposition reveals how the curvature influences the error dynamics along different principal directions.

GNN (4) is structurally simple but lacks a mechanism to compensate for the dynamic changes of the optimization landscape, a deficiency known to lead to an unavoidable lagging error (Zhang et al., 2009). The following theorem characterizes the lagging error for the TVNO problem (1).

Theorem 1. *For the TVNO problem (1), the error of GNN (4) generally does not converge to zero. Instead, due to the drift term induced by time variation, the error is confined to a residual region whose size is determined by the ratio between the drift magnitude and the minimum curvature of the system.*

Proof. Consider the energy function $L(t)$ as a Lyapunov candidate:

$$L(t) = \frac{1}{2}\sum_{i=1}^{n}\varepsilon_i^2(t). \tag{5}$$

Differentiating (5) with respect to time yields

$$\begin{aligned}\dot{L}(t) &= \sum_{i=1}^{n}\varepsilon_i(t)\dot{\varepsilon}_i(t) \\ &= \sum_{i=1}^{n}\varepsilon_i(t)\left(\sum_{j=1}^{n}\frac{\partial\varepsilon_i}{\partial x_j}\dot{x}_j(t) + \frac{\partial\varepsilon_i}{\partial t}\right).\end{aligned} \tag{6}$$

Substituting GNN (4) into (6) yields

$$\dot{L}(t) = \sum_{i=1}^{n}\varepsilon_i(t)\left(-\gamma\sum_{j=1}^{n}\sum_{k=1}^{n}\mathcal{D}_{ij}\mathcal{D}_{kj}\varepsilon_k(t) + \phi_i(t)\right),$$

where $\phi_i(t) \triangleq \partial\varepsilon_i/\partial t$ denotes the drift term induced by time variation. Applying the Rayleigh quotient inequality to the quadratic term and the Cauchy–Schwarz inequality to the drift term yields

$$\begin{aligned}\dot{L}(t) &\le -\gamma\lambda_{min}^2(t)\|\boldsymbol{\varepsilon}\|_2^2 + \sum_{i=1}^{n}\varepsilon_i\phi_i \\ &\le -\|\boldsymbol{\varepsilon}\|_2\left(\gamma\lambda_{min}^2(t)\|\boldsymbol{\varepsilon}\|_2 - \|\boldsymbol{\phi}\|_2\right),\end{aligned}$$

According to Lyapunov stability theory, $\dot{L}(t)$ is strictly negative definite and the error norm satisfies:

$$\|\boldsymbol{\varepsilon}(t)\|_2 > \frac{\|\boldsymbol{\phi}(t)\|_2}{\gamma\lambda_{min}^2(t)}. \tag{7}$$

This implies that GNN (4) generally does not converge to the origin. Instead, its error is driven into a residual domain determined by the drift term and the curvature lower bound. Accordingly, the residual error bound is given by

$$\limsup_{t\to\infty}\|\boldsymbol{\varepsilon}(t)\|_2 \le \frac{\|\boldsymbol{\phi}(t)\|_2}{\gamma\lambda_{min}^2(t)}. \tag{8}$$

□

Remark 1. Theorem 1 shows that the lagging error is determined by the ratio $\|\boldsymbol{\phi}(t)\|_2/\gamma\lambda_{\min}^2(t)$, which reflects two distinct sources of degradation. The numerator $\|\boldsymbol{\phi}(t)\|_2$ measures the drift induced by time variation and cannot be suppressed by increasing γ alone. The denominator $\gamma\lambda_{\min}^2(t)$ characterizes the feedback correction capability and becomes weak under weak-curvature conditions. Hence, eliminating the lagging error requires **(i)** compensating for the drift term $\boldsymbol{\phi}(t)$ and **(ii)** enhancing the denominator when $\lambda_{\min}(t)$ is small.

3. Architecture and design of AGCNN

3.1. Feedforward velocity compensation mechanism

In this subsection, we address the drift induced by $\boldsymbol{\phi}(t)$. To this end, a feedforward velocity compensation mechanism is introduced by adding a compensation term $z_i(t)$ to the dynamics of the ith neuron:

$$\dot{x}_i(t) = -\gamma\sum_{j=1}^{n}\mathcal{D}_{ji}(\boldsymbol{x}(t),t)\varepsilon_j(t) + z_i(t). \tag{9}$$

To ensure that the system achieves zero steady-state error, the $z_i(t)$ is synthesized to track the time-varying optimal trajectory. For exact tracking, the desired compensation velocity should satisfy

$$\dot{x}_i(t) = z_i(t). \tag{10}$$

To preserve the zero error condition, the total time derivative of the error must vanish identically. By expanding $\dot{\varepsilon}_i(t)$ according to the chain rule, we obtain the consistency condition:

$$\dot{\varepsilon}_i(t) = \sum_{j=1}^{n}\mathcal{D}_{ij}(t)\dot{x}_j(t) + \phi_i(t) = 0. \tag{11}$$

Substituting the desired compensation velocity (10) into the consistency condition (11) yields a system of linear equations with respect to the compensation terms:

$$\sum_{j=1}^{n}\mathcal{D}_{ij}(t)z_j(t) + \phi_i(t) = 0. \tag{12}$$

By solving Eq. (12), the explicit expression for the velocity compensation term $z_i(t)$ is obtained as

$$z_i(t) = -\sum_{k=1}^{n}\mathcal{W}_{ik}(t)\phi_k(t), \tag{13}$$

where $\mathcal{W}_{ik}(t)$ denotes the (i,k)th element of the pseudoinverse Hessian matrix $\mathcal{D}^{\dagger}(t)$. This term explicitly compensates for the inherent lagging error of GNN (4).

3.2. Adaptive spectral compensation mechanism

Although the feedforward compensation term theoretically eliminates the drift-induced lagging error by explicitly canceling $\boldsymbol{\phi}(t)$, its effectiveness depends on the numerical stability of the pseudoinverse Hessian matrix $\mathcal{D}^{\dagger}(t)$. Specifically, the magnitude of the compensation term satisfies

$$\|\boldsymbol{z}(t)\|_2 \le \|\mathcal{D}^{\dagger}(t)\|_2\|\boldsymbol{\phi}(t)\|_2, \tag{14}$$

which shows that the magnitude of the compensation term scales with $\|\mathcal{D}^{\dagger}(t)\|_2 = \lambda_{\min}^{-1}(t)$, so that the feedforward channel becomes increasingly sensitive to perturbations as the curvature weakens.

Consequently, the feedforward compensation term alone cannot guarantee complete lagging error elimination under weak-curvature conditions. Conversely, without the feedforward term $\boldsymbol{z}(t)$, the drift $\boldsymbol{\phi}(t)$ remains unaddressed regardless of the feedback gain. Hence, spectral scaling alone is also insufficient.

Therefore, the two mechanisms must operate jointly: the feedforward compensation mechanism targets the drift numerator $\|\boldsymbol{\phi}(t)\|_2$, the

adaptive spectral compensation mechanism, introduced below, compensates for the weakening of the curvature denominator.

To address the spectral degradation problem, an adaptive spectral compensation factor is introduced as

$$\Xi(t) = \max_i (\lambda_i(t) - \epsilon)^{-2}, \tag{15}$$

where $\epsilon \geq 0$ is a fixed spectral margin satisfying

$$0 \leq \epsilon < \inf_{t\geq 0} \lambda_{\min}(t),$$

which ensures that $\Xi(t)$ remains finite and well-defined for all $t \geq 0$. Since $D(t)$ is positive definite, the maximum in (15) is attained at $\lambda_{\min}(t)$. Hence,

$$\Xi(t) = \frac{1}{\big(\lambda_{\min}(t) - \epsilon\big)^2}.$$

Therefore, $\Xi(t)$ increases monotonically as $\lambda_{\min}(t)$ decreases, thereby adaptively amplifying the feedback gain when the local curvature becomes weak.

The rationale for this design can be understood from Theorem 1. For the uncompensated GNN (4), the lagging error bound satisfies

$$\|\varepsilon(t)\|_2 \leq \frac{\|\dot{\boldsymbol{\phi}}(t)\|_2}{\gamma \lambda_{\min}^2(t)} \leq \frac{\Xi(t)}{\gamma}\|\dot{\boldsymbol{\phi}}(t)\|_2,$$

which shows that a decrease in curvature directly amplifies the lagging error. Furthermore, for the feedforward compensation term $\boldsymbol{z}(t) = -D^{\dagger}(t)\dot{\boldsymbol{\phi}}(t)$, the bound in (14) can be tightened as

$$\|\boldsymbol{z}(t)\|_2^2 \leq \|D^{\dagger}(t)\|_2^2 \|\dot{\boldsymbol{\phi}}(t)\|_2^2 \leq \Xi(t)\|\dot{\boldsymbol{\phi}}(t)\|_2^2,$$

where the second inequality follows from the identity $\|D^{\dagger}(t)\|_2 = 1/\lambda_{\min}(t) \leq 1/(\lambda_{\min}(t) - \epsilon) = \sqrt{\Xi(t)}$. This inequality explicitly links the sensitivity of the compensation channel to $\Xi(t)$: when $\Xi(t)$ becomes large, the compensation term becomes more vulnerable to perturbations, and the adaptive factor correspondingly reinforces the feedback channel.

3.3. EPMAF

After eliminating the inherent lag error, to further accelerate the convergence of the dynamical system and enhance robustness against noise, a nonlinear AF is typically applied to the error. Existing AFs, such as the Power-Sigmoid (Zhang et al., 2013), Sign-Bi-Power (Li et al., 2012), and Hyperbolic Sine (Zhang et al., 2012), can improve convergence performance to some extent, but they usually exhibit an inherent trade-off. Specifically, high-gain AFs provide fast error suppression in the transient stage, yet they often become overly rigid and noise-sensitive near the equilibrium; in contrast, low-gain AFs are numerically smoother but may lead to sluggish convergence.

To balance these conflicting requirements, the EPMAF is defined as

$$\vartheta_{\text{epm}}(\varepsilon_i) = \frac{(|\varepsilon_i|^{0.5} + \varsigma)^{(|\varepsilon_i|^{0.5}+\varsigma)}}{\ln(|\varepsilon_i|^{0.5} + \varsigma) + 1}\ \operatorname{sgn}(\varepsilon_i)\,|\varepsilon_i|^{0.5}. \tag{16}$$

where $\varsigma \geq 1$ is a tunable shape parameter.

Specifically, when $|\varepsilon_i|$ is relatively large, the power-exponential term strengthens the nonlinear correction effect and accelerates transient error decay. Meanwhile, the logarithmic denominator moderates the growth of the activation output and helps avoid overly aggressive responses. As $|\varepsilon_i|$ approaches zero, the fractional sign-based structure preserves sufficient sensitivity to small deviations while alleviating the stiffness commonly associated with conventional high-gain AFs.

Lemma 1. *For any $\varsigma \geq 1$, the activation ϑ_{epm} defined in (16) satisfies:*

(i) it is odd, strictly increasing, and $\vartheta_{\text{epm}}(0) = 0$;

(ii) there exists $\kappa \geq 1$ such that $\vartheta_{\text{epm}}(\varepsilon)\,\varepsilon \geq \kappa\,\varepsilon^2$ for all $\varepsilon \in \mathbb{R}$, that is, ϑ_{epm} lies in the sector $[\kappa, \infty)$. Equivalently $\|\boldsymbol{\vartheta}_{\text{epm}}(\boldsymbol{\varepsilon})\|_2 \geq \kappa\|\boldsymbol{\varepsilon}\|_2$, where κ is determined by ς and is not a design parameter.

Proof. (i) follows from the factor $\operatorname{sgn}(\varepsilon)|\varepsilon|^{0.5}$ together with the positivity of $(|\varepsilon|^{0.5} + \varsigma)^{(|\varepsilon|^{0.5}+\varsigma)}$ and of $\ln(|\varepsilon|^{0.5} + \varsigma) + 1$ for $\varsigma \geq 1$. With $w = |\varepsilon|^{0.5} + \varsigma$ and $f(w) = w^w/(\ln w + 1)$, $f'(w) = w^w[(\ln w + 1)^2 - 1/w]/(\ln w + 1)^2 > 0$ for $w > 1$, so ϑ_{epm} is a product of positive increasing functions and is strictly increasing. For (ii), let $\rho(\varepsilon) = \vartheta_{\text{epm}}(\varepsilon)/\varepsilon$ on $\varepsilon > 0$; the case $\varepsilon < 0$ is identical by oddness. As $\varepsilon \to 0^+$, $\vartheta_{\text{epm}}(\varepsilon) \sim C\varepsilon^{0.5}$ with $C = \varsigma^{\varsigma}/(\ln\varsigma + 1)$, so $\rho(\varepsilon) \to \infty$; as $\varepsilon \to \infty$, the power-exponential factor grows faster than any polynomial, so $\rho(\varepsilon) \to \infty$ as well. Being continuous and positive on $(0, \infty)$, ρ attains a strictly positive infimum κ. Moreover $\rho = w^w/[(\ln w + 1)(w - \varsigma)]$, and since $\ln w \leq w - 1$ and $\varsigma \geq 1$, $\rho \geq w^{w-1}/(w - 1) > 1$; hence $\kappa \geq 1$. Numerically, $\kappa \geq 2.27$ for the values of ς considered in this paper. □

3.4. AGCNN

By integrating the aforementioned mechanisms, AGCNN is formulated as follows:

$$\dot{\boldsymbol{x}}(t) = -\gamma\Xi(t)D(\boldsymbol{x}(t), t)\boldsymbol{\vartheta}_{\text{epm}}(\boldsymbol{\varepsilon}(t)) - \mathcal{D}^{\dagger}(\boldsymbol{x}(t), t)\dot{\boldsymbol{\phi}}(t). \tag{17}$$

The network consists of n coupled neurons, where the state of the ith neuron $x_i(t)$ evolves according to the combination of lateral feedback and feedforward signals.

The dynamics of the ith neuron in AGCNN (17) are given by

$$\dot{x}_i(t) = -\gamma\Xi(t)\sum_{j=1}^{n}\mathcal{D}_{ji}(t)\vartheta_{\text{epm}}(\varepsilon_j(t)) - \sum_{k=1}^{n}\mathcal{W}_{ik}(t)\dot{\phi}_k(t), \tag{18}$$

for $i = 1, \ldots, n$. The computational diagram of AGCNN (17) is shown in Fig. 1, and its implementation at each sampling instant is summarized in Algorithm 1.

Algorithm 1 Pseudocode of AGCNN for solving the TVNO problem.

Require: initial state $\boldsymbol{x}(0)$; parameters $\gamma > 0$, $\varsigma \geq 1$, $\epsilon \geq 0$; horizon T; sampling interval τ

Ensure: state trajectory $\boldsymbol{x}(t)$

$t \leftarrow 0$, $\boldsymbol{x} \leftarrow \boldsymbol{x}(0)$

while $t \leq T$ **do**

  evaluate $\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{x}}\mathcal{Q}(\boldsymbol{x}, t)$, $\boldsymbol{\varepsilon} \leftarrow \boldsymbol{g}$, $\dot{\boldsymbol{\phi}} \leftarrow \partial\boldsymbol{g}/\partial t$, $\mathcal{D} \leftarrow \nabla_{\boldsymbol{x}}^2\mathcal{Q}(\boldsymbol{x}, t)$

  compute the symmetric eigendecomposition $\mathcal{D} = Q\Lambda Q^{\mathrm{T}}$

  $\lambda_{\min} \leftarrow \min \operatorname{diag}(\Lambda)$, $\Xi \leftarrow (\lambda_{\min} - \epsilon)^{-2}$

  evaluate $\boldsymbol{\vartheta}_{\text{epm}}(\boldsymbol{\varepsilon})$ componentwise by (16)

  $\boldsymbol{z} \leftarrow -Q\big(\Lambda^{-1}(Q^{\mathrm{T}}\dot{\boldsymbol{\phi}})\big)$ {$\boldsymbol{z} = -\mathcal{D}^{\dagger}\dot{\boldsymbol{\phi}}$ as in (13), evaluated without forming $\mathcal{D}^{\dagger}$}

  $\dot{\boldsymbol{x}} \leftarrow -\gamma\Xi\,\mathcal{D}\,\boldsymbol{\vartheta}_{\text{epm}}(\boldsymbol{\varepsilon}) + \boldsymbol{z}$

  $\boldsymbol{x} \leftarrow \boldsymbol{x} + \tau\dot{\boldsymbol{x}}$, $t \leftarrow t + \tau$

end while

Unlike GNN (4), AGCNN (17) is not inversion-free: the feedforward channel requires an inverse-Hessian action at each instant. This trades the inversion-free simplicity of GNN (4) for explicit cancelation of the drift.

3.5. Computational complexity

To analyze the computational complexity of AGCNN (17), the flop-counting conventions of Boyd and Vandenberghe (2004) are adopted as follows: 1) a scalar–vector product of length n costs n flops; 2) an $n \times n$ matrix–vector product costs $n(2n - 1)$ flops; and 3) an $n \times n$ matrix (pseudo) inversion costs n^3 flops. Specifically, at each time instant, $\boldsymbol{\vartheta}_{\text{epm}}(\boldsymbol{\varepsilon})$ costs $c_{\vartheta}n$ flops, where c_{ϑ} denotes the constant cost of a single EPMAF evaluation, and the feedback product $\mathcal{D}\boldsymbol{\vartheta}_{\text{epm}}(\boldsymbol{\varepsilon})$ costs $2n^2 - n$ flops. Notably, the adaptive spectral factor Ξ and the feedforward term $-\mathcal{D}^{\dagger}\dot{\boldsymbol{\phi}}$ share a single symmetric eigendecomposition $\mathcal{D} = Q\Lambda Q^{\mathrm{T}}$, with Q orthogonal and Λ diagonal, which costs $\mathcal{O}(n^3)$ flops and simultaneously yields $\lambda_{\min} = \min \operatorname{diag}(\Lambda)$ for Ξ (a further 3 flops) together with $\mathcal{D}^{\dagger} = Q\Lambda^{-1}Q^{\mathrm{T}}$.

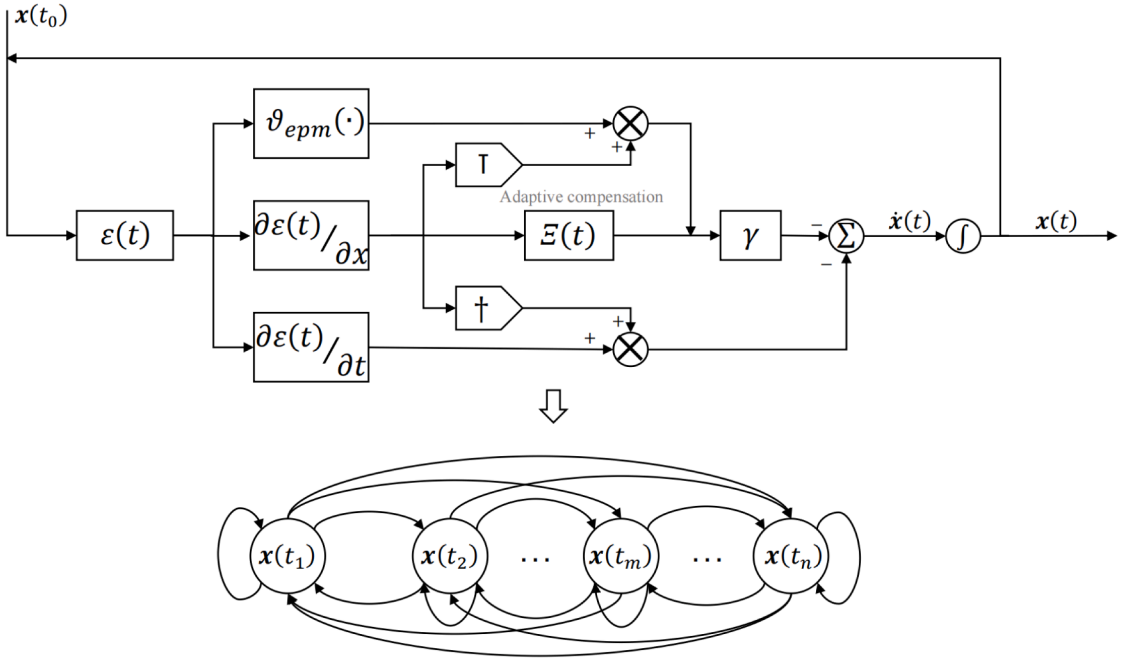


Fig. 1. Computational diagram of AGCNN (17).

In view of this decomposition, $D^{\dagger}\boldsymbol{\phi}$ costs $4n^2 - n$ flops. A Cholesky factorization ($\approx n^3/3$ flops) would suffice if only $D^{\dagger}$ were required; since Ξ additionally requires $\lambda_{\min}$, the symmetric eigendecomposition is retained, as it supplies both quantities from a single factorization. In summary, AGCNN (17) consumes $n^3 + 6n^2 + (c_{\vartheta} + 1)n + 3 = \mathcal{O}(n^3)$ floating-point operations at each time instant.

For problems of low to moderate dimension the cost does not restrict real-time operation: for $n \le 10$, for instance, the total remains on the order of 10^3 flops per instant, a small fraction of the budget available at a kilohertz sampling rate on a standard processor. In terms of scalability, the per-instant cost is dominated by the single symmetric eigendecomposition and therefore grows as $\mathcal{O}(n^3)$. For high-dimensional problems (e.g., $n > 100$), particularly when D is dense and rapidly time-varying, this term becomes comparable to the sampling period at kilohertz rates and would restrict real-time operation. AGCNN (17) is thus intended for TVNO problems of low to moderate dimension under stringent real-time constraints, and its extension to large-scale settings is left to future work.

4. Theoretical analyses

4.1. Convergence analysis

Under the joint action of the feedforward velocity compensation mechanism and the adaptive spectral compensation mechanism, the lagging error is eliminated and global exponential convergence is achieved. To isolate the contribution of these two mechanisms from that of the activation, consider first the intermediate system in which $\boldsymbol{\vartheta}_{\text{epm}}(\cdot)$ is replaced by the error itself:

$$\dot{x}_i(t) = -\gamma\,\Xi(t)\sum_{j=1}^{n} D_{ji}(t)\,\varepsilon_j(t) - \sum_{k=1}^{n} \mathcal{W}_{ik}(t)\,\phi_k(t). \tag{19}$$

Theorem 2. *The error of system (19) converges to zero globally and exponentially, namely,*

$$\|\boldsymbol{\varepsilon}(t)\|_2 \le \|\boldsymbol{\varepsilon}(0)\|_2\, e^{-\gamma t}, \quad \forall\, t \ge 0. \tag{20}$$

Proof. Consider $L(t) = \frac{1}{2}\|\boldsymbol{\varepsilon}(t)\|_2^2$ as a Lyapunov candidate. The feedforward term cancels $\boldsymbol{\phi}(t)$ through the consistency condition. Therefore,

$$\dot{L}(t) = -\gamma\,\Xi(t)\,\boldsymbol{\varepsilon}^{\top}(t)\,D(t)D^{\top}(t)\,\boldsymbol{\varepsilon}(t).$$

By the spectral property of $D(t)D^{\top}(t)$, one has

$$\dot{L}(t) \le -\gamma\,\Xi(t)\,\lambda_{\min}^2(t)\,\|\boldsymbol{\varepsilon}(t)\|_2^2.$$

Since

$$\Xi(t)\,\lambda_{\min}^2(t) = \frac{\lambda_{\min}^2(t)}{(\lambda_{\min}(t) - \epsilon)^2} \ge 1,$$

it follows that

$$\dot{L}(t) \le -2\gamma\,L(t).$$

□

Remark 2. Theorem 2 shows that the feedforward velocity compensation mechanism and the adaptive spectral compensation mechanism jointly eliminate the lagging error characterized in Theorem 1. Specifically, the feedforward term compensates for the drift $\boldsymbol{\phi}(t)$, while $\Xi(t)$ prevents the convergence rate from degrading when $\lambda_{\min}(t)$ is small.

Theorem 3. *Under the assumption of Section 2, the error of AGCNN (17) converges to zero globally in finite time, with*

$$\tilde{t} \le \frac{2n\left(\varsigma^{-\varsigma} - (\tilde{\varepsilon}^{0.5}(0) + \varsigma)^{-(\tilde{\varepsilon}^{0.5}(0)+\varsigma)}\right)}{\gamma},$$

where $\tilde{\varepsilon}(0)$ denotes the error component of largest initial magnitude. Moreover, regardless of the initial state, the convergence time admits the absolute upper bound $2n\varsigma^{-\varsigma}/\gamma$.

Proof. Let $\upsilon(E) = \int_0^E \vartheta_{\text{epm}}(s)\,\mathrm{d}s$ and consider

$$V(\boldsymbol{\varepsilon}) = \sum_{i=1}^{n} \upsilon(|\varepsilon_i|).$$

By Lemma 1(i), V is positive definite, continuously differentiable and radially unbounded, and is therefore a valid Lyapunov candidate.

Substituting AGCNN (17) into $\dot{\boldsymbol{\varepsilon}} = D\dot{\boldsymbol{x}} + \dot{\boldsymbol{\phi}}$, the feedforward term cancels the drift exactly and

$$\dot{\boldsymbol{\varepsilon}} = D\left(-\gamma\Xi D\boldsymbol{\vartheta}_{\text{epm}} - D^{\dagger}\boldsymbol{\phi}\right) + \dot{\boldsymbol{\phi}} = -\gamma\Xi\,D^2\boldsymbol{\vartheta}_{\text{epm}}. \tag{21}$$

Differentiating V along (21) gives $\dot{V} = \boldsymbol{\vartheta}_{\text{epm}}^{\top}\dot{\boldsymbol{\varepsilon}} = -\gamma\Xi\,\boldsymbol{\vartheta}_{\text{epm}}^{\top}D^2\boldsymbol{\vartheta}_{\text{epm}}$. Since D is symmetric, D^2 is symmetric positive semidefinite with eigenvalues λ_i^2, so the Rayleigh quotient inequality together with $\Xi\lambda_{\min}^2 \ge 1$ yields

$$\dot{V} \le -\gamma\,\Xi\lambda_{\min}^2\,\|\boldsymbol{\vartheta}_{\text{epm}}\|_2^2 \le -\gamma\|\boldsymbol{\vartheta}_{\text{epm}}\|_2^2. \tag{22}$$

The coupling matrix D^2 is retained in full, so no decoupling of the neurons is invoked.

Let $E = \max_i |\varepsilon_i|$. By Lemma 1(i), ϑ_{epm} is positive on $(0,\infty)$, so υ is continuous and strictly increasing on $[0,\infty)$ with $\upsilon(0)=0$ and its inverse υ^{-1} is well defined. Since every term of $\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2^2$ is nonnegative, $\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2^2 \geq \vartheta_{\mathrm{epm}}^2(E)$, while $V \leq n\,\upsilon(E)$ since υ is increasing, that is, $E \geq \upsilon^{-1}(V/n)$. Substituting these two estimates into (22) and using the monotonicity of ϑ_{epm} yields the scalar differential inequality

$$\dot{V} \leq -\gamma\, \vartheta_{\mathrm{epm}}^2\big(\upsilon^{-1}(V/n)\big),$$

in which the coupling has been reduced to a comparison inequality for V itself rather than for any individual neuron.

Separating variables and changing the integration variable by $V = n\,\upsilon(s)$, so that $\mathrm{d}V = n\,\vartheta_{\mathrm{epm}}(s)\,\mathrm{d}s$ and $\upsilon^{-1}(V/n) = s$, the time to reach the origin satisfies

$$\tilde{t} \leq \int_0^{V(0)} \frac{\mathrm{d}V}{\gamma\,\vartheta_{\mathrm{epm}}^2\big(\upsilon^{-1}(V/n)\big)} = \frac{n}{\gamma}\int_0^{s_0} \frac{\mathrm{d}s}{\vartheta_{\mathrm{epm}}(s)}, \qquad s_0 = \upsilon^{-1}\big(V(0)/n\big).$$

Since $V(0) \leq n\,\upsilon(E(0))$ with $E(0) = \max_i |\varepsilon_i(0)|$, one has $s_0 \leq E(0)$, and as the integrand is positive the upper limit may be enlarged to $E(0)$, which gives $\tilde{t} \leq (n/\gamma)\int_0^{E(0)} \mathrm{d}s/\vartheta_{\mathrm{epm}}(s)$.

Evaluating the integral,

$$\begin{aligned}\int_0^{E} \frac{\mathrm{d}E}{\vartheta_{\mathrm{epm}}(E)} &= \int_0^{E} \frac{\big(\ln(E^{0.5}+\varsigma)+1\big)E^{-0.5}}{(E^{0.5}+\varsigma)^{(E^{0.5}+\varsigma)}}\,\mathrm{d}E \\ &= 2\Big(\varsigma^{-\varsigma} - (E^{0.5}+\varsigma)^{-(E^{0.5}+\varsigma)}\Big).\end{aligned}$$

Since $E(0) = |\tilde{\varepsilon}(0)|$, the stated bound follows. Letting $E(0) \to \infty$ yields the absolute bound $2n\varsigma^{-\varsigma}/\gamma$. □

Remark 3. The quantity $\tilde{t}$ provides an upper bound on the convergence time, rather than the exact convergence instant.

4.2. Robustness analysis

In practical scenarios, time-varying operations often lead to the accumulation of noise and rounding errors. Although these perturbations may be subtle, they cannot be neglected, as they may gradually drive the system to an undesirable state.

To facilitate the subsequent analysis, the noise is assumed to satisfy

$$\boldsymbol{\eta}(t) \in \mathbb{R}^n, \quad \|\boldsymbol{\eta}(t)\|_2 \leq \varpi,$$

where $\varpi \geq 0$, and $\boldsymbol{\eta}(t)$ represents both periodic and aperiodic bounded disturbances.

Theorem 4. *In the presence of the bounded additive noise $\boldsymbol{\eta}(t)$ with $\|\boldsymbol{\eta}(t)\|_2 \leq \varpi$, the error of AGCNN (17) is globally ultimately bounded, and its steady-state norm converges within a residual set proportional to the noise level:*

$$\limsup_{t\to\infty} \|\boldsymbol{\varepsilon}(t)\|_2 \leq \frac{\varpi}{\gamma}.$$

Proof. With the perturbation entering additively, (21) becomes $\dot{\boldsymbol{\varepsilon}} = -\gamma\Xi D^2\boldsymbol{\vartheta}_{\mathrm{epm}} + \boldsymbol{\eta}$. Differentiating V along this system and using (22) together with the Cauchy–Schwarz inequality,

$$\dot{V} \leq -\gamma\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2^2 + \|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2\|\boldsymbol{\eta}\|_2 \leq -\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2\big(\gamma\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2 - \varpi\big),$$

so $\dot{V} < 0$ whenever $\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2 > \varpi/\gamma$. Every trajectory therefore enters, and thereafter remains in, the set $\{\boldsymbol{\varepsilon} : \|\boldsymbol{\vartheta}_{\mathrm{epm}}(\boldsymbol{\varepsilon})\|_2 \leq \varpi/\gamma\}$. By Lemma 1(ii), $\|\boldsymbol{\vartheta}_{\mathrm{epm}}\|_2 \geq \kappa\|\boldsymbol{\varepsilon}\|_2$, whence

$$\limsup_{t\to\infty} \|\boldsymbol{\varepsilon}(t)\|_2 \leq \frac{\varpi}{\gamma\kappa} \leq \frac{\varpi}{\gamma},$$

the last step following from $\kappa \geq 1$. No linearization of the activation is used; the linear bound is recovered because ϑ_{epm} lies above the identity sector, so that treating it as linear understates its restoring action. □

Table 1
Numerical results of AGCNN (17) under different values of ς and different initialization intervals.

Parameter	$\tilde{t}$ (s)	Initialization interval	CT (s)	MSSRE	ASSRE
$\varsigma = 1$	8	(0, 10)	1.17×10^{0}	2.58×10^{-4}	1.24×10^{-5}
		(10, 20)	1.77×10^{0}	1.66×10^{-4}	8.52×10^{-6}
		(20, 30)	1.90×10^{0}	2.34×10^{-4}	1.47×10^{-5}
		(30, 40)	1.95×10^{0}	9.30×10^{-5}	9.32×10^{-6}
$\varsigma = 1.5$	4.35	(0, 10)	7.27×10^{-1}	8.88×10^{-5}	5.48×10^{-6}
		(10, 20)	9.32×10^{-1}	3.06×10^{-4}	3.30×10^{-5}
		(20, 30)	1.03×10^{0}	1.22×10^{-4}	7.22×10^{-6}
		(30, 40)	1.05×10^{0}	8.95×10^{-4}	2.59×10^{-5}
$\varsigma = 2$	2	(0, 10)	3.37×10^{-1}	8.81×10^{-4}	2.04×10^{-5}
		(10, 20)	4.64×10^{-1}	1.56×10^{-4}	8.48×10^{-6}
		(20, 30)	4.79×10^{-1}	9.79×10^{-5}	6.87×10^{-6}
		(30, 40)	4.85×10^{-1}	5.27×10^{-4}	1.31×10^{-5}
$\varsigma = 2.5$	0.81	(0, 10)	1.38×10^{-1}	2.5×10^{-3}	1.76×10^{-5}
		(10, 20)	1.83×10^{-1}	1.17×10^{-3}	1.11×10^{-5}
		(20, 30)	1.95×10^{-1}	1.71×10^{-4}	8.01×10^{-6}
		(30, 40)	1.95×10^{-1}	2.83×10^{-4}	6.88×10^{-6}

5. Numerical simulations

This section demonstrates, through numerical simulations, the effectiveness and robustness of AGCNN (17) in solving the TVNO problem under various conditions.

5.1. Effectiveness evaluation

Example 1. Consider the following TVNO problem:

$$\min_{\boldsymbol{x}(t)\in\mathbb{R}^4} \mathcal{Q}(\boldsymbol{x}(t), t), \tag{23}$$

where $\boldsymbol{x}(t) = [x_1, x_2, x_3, x_4]^{\mathrm{T}}$, and $\mathcal{Q}(\boldsymbol{x}(t), t)$ is defined as

$$\begin{aligned}\mathcal{Q}(\boldsymbol{x}(t), t) = {} & (x_1 + t)^2 + (x_2 - t)^2 + (x_3 - e^{-t})^2 \\ & + (x_4 - \ln(t + 0.1))^2 + (x_1 + \sin(2t))x_3 \\ & + (x_4 - \cos(t))x_2.\end{aligned}$$

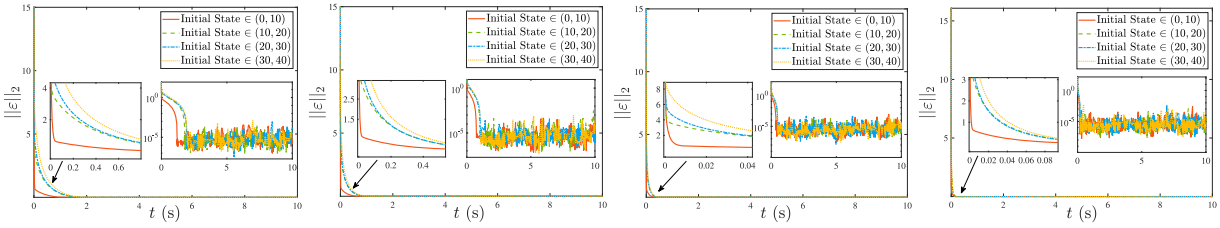
To investigate the impact of different initial conditions on AGCNN (17), the initial state $\boldsymbol{x}(0)$ is randomly sampled from four different intervals. In this experiment, we focus on the effect of the tunable shape parameter ς on AGCNN (17). Specifically, ς is set to 1, 1.5, 2, and 2.5, while γ is fixed at 1. The simulation horizon is 10 s. Three metrics are adopted: the convergence time (CT), the maximum steady-state residual error (MSSRE), and the average steady-state residual error (ASSRE). Their mathematical definitions are as follows:

$$\begin{cases}\mathrm{MSSRE} = \max\limits_{t\in\mathcal{T}_{\mathrm{ss}}} \|\boldsymbol{\varepsilon}(t)\|_2, \\ \mathrm{ASSRE} = \frac{1}{|\mathcal{T}_{\mathrm{ss}}|}\int_{\mathcal{T}_{\mathrm{ss}}} \|\boldsymbol{\varepsilon}(t)\|_2\, dt, \\ \mathrm{CT} \quad = \inf\big\{\, t \geq 0 \;\big|\; \|\boldsymbol{\varepsilon}(t)\|_2 \leq \mathrm{MSSRE}\,\big\},\end{cases} \tag{24}$$

where T denotes the simulation horizon and $\mathcal{T}_{\mathrm{ss}} = [5T/6, T]$ is taken as the steady-state interval. MSSRE and ASSRE respectively measure the accuracy and the numerical stability of the converged solution, while CT measures the convergence speed.

As shown in Fig. 2, the $|\boldsymbol{\varepsilon}(t)|_2$ rapidly converges to zero under all four initialization intervals, thereby providing numerical evidence of the global convergence behavior of AGCNN (17). A comparative analysis of Fig. 2(a)–(d) reveals a clear correlation between ς and the convergence rate. Specifically, as ς increases from 1 to 2.5, the error decays more rapidly, indicating a significantly faster convergence process.

This observation is quantitatively corroborated by the data in Table 1. The theoretical upper bound on the convergence time, denoted by $\tilde{t}$, decreases monotonically from 8 s to 0.81 s as ς increases. Correspondingly, the recorded CT exhibits a consistent downward trend. For instance, when $\varsigma = 2.5$, the average CT drops to the order of 10^{-1}, s, which



(a) $\varsigma = 1$ (b) $\varsigma = 1.5$ (c) $\varsigma = 2$ (d) $\varsigma = 2.5$

Fig. 2. Performance of AGCNN (17). (a)–(d) Error trajectories and their logarithmic plots generated by AGCNN (17) under different values of ς and four different initialization intervals.

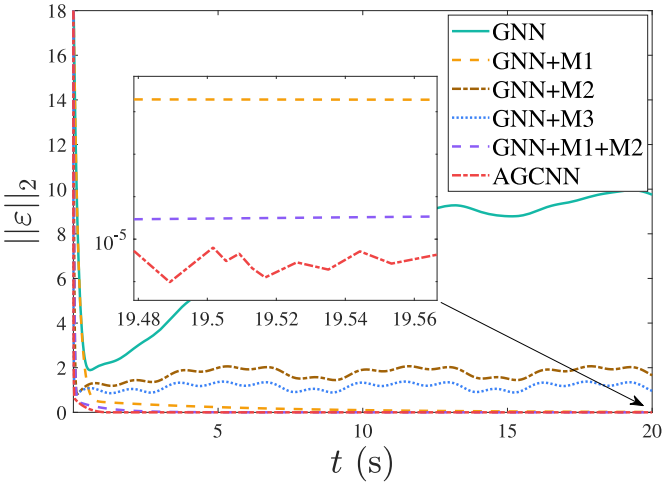


Fig. 3. Ablation results of different mechanism combinations on Example 1.

Table 2
Ablation results of GNN (4) on Example 1 under different combinations of mechanisms, where M1, M2, and M3 denote the feedforward velocity compensation mechanism, the adaptive spectral compensation mechanism, and EPMAF, respectively.

M1	M2	M3	CT (s)	MSSRE	ASSRE
			9.90×10^{-2}	9.95×10^{0}	9.59×10^{0}
✓			1.58×10^{1}	6.27×10^{-1}	4.57×10^{-1}
	✓		6.47×10^{-2}	2.07×10^{0}	1.88×10^{0}
		✓	2.70×10^{-2}	1.38×10^{0}	1.19×10^{0}
✓	✓		8.84×10^{0}	8.05×10^{-4}	2.89×10^{-5}
✓	✓	✓	$\mathbf{1.14 \times 10^{0}}$	$\mathbf{6.75 \times 10^{-4}}$	$\mathbf{2.22 \times 10^{-5}}$

remains below the theoretical bound $\tilde{t}$, thereby validating the conclusion of Theorem 3. This confirms that AGCNN (17) possesses the finite-time convergence property and that ς effectively regulates the convergence rate.

Furthermore, regarding the solution accuracy, Table 1 indicates that the model maintains high precision across varying conditions. The MSSRE generally remains on the order of 10^{-4}, and the ASSRE remains on the order of 10^{-5} to 10^{-6}. These results suggest that while increasing ς accelerates convergence, it does not compromise the solution accuracy.

5.2. Ablation study

To evaluate the contribution of each mechanism, an ablation study is conducted on Example 1 with $\gamma = 1$ and $\varsigma = 1$. The simulation horizon is 20 s. Six variants are considered: GNN (4), GNN (4) + M1, GNN (4) + M2, GNN (4) + M3, GNN (4) + M1 + M2, and AGCNN (17), where M1, M2, and M3 denote the feedforward velocity compensation mechanism, the adaptive spectral compensation mechanism, and EPMAF, respectively.

As shown in Fig. 3 and Table 2, the ablation results clearly reveal the individual roles of the three mechanisms in AGCNN (17). The variants GNN (4) + M1, GNN (4) + M2, and GNN (4) + M3 all retain MSSRE at the order of 10^{-1} to 10^{0}, indicating that each individual mechanism can reduce the residual to some extent but cannot eliminate the intrinsic lagging identified in Theorem 1.

Compared with GNN (4), GNN (4) + M1 reduces the MSSRE to 6.27×10^{-1}, but its CT increases to 1.58×10^{1} s. This behavior is consistent with the analysis in Section 3.2: when $\lambda_{\min}(t)$ becomes small, $\|D^{\dagger}(t)\|_2$ may become large, thereby amplifying perturbations and introducing numerical stiffness. This result indicates that the feedforward velocity compensation mechanism alone is insufficient and requires the adaptive spectral compensation mechanism for stable operation.

GNN (4) + M1 + M2 achieves MSSRE of 8.05×10^{-4} and ASSRE of 2.89×10^{-5}, representing an improvement of approximately four orders of magnitude over GNN (4). This result is consistent with Theorem 2: the feedforward term compensates the drift $\boldsymbol{\phi}(t)$, while the adaptive spectral factor stabilizes the correction channel under weak-curvature conditions. As a consequence, the lagging error is effectively eliminated and the error is driven to the theoretical-solution level.

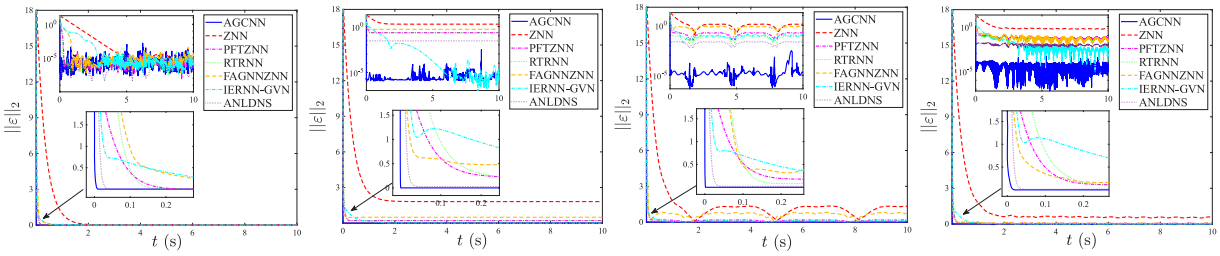
After further incorporating EPMAF (16), AGCNN (17) reduces the CT from 8.84 s to 1.14 s, corresponding to a 7.7× speedup, while maintaining an ASSRE of 2.22×10^{-5}. This result shows that EPMAF (16) mainly improves the transient convergence rate, rather than being the primary mechanism for eliminating the lagging error.

5.3. Robustness validation

To further validate the robustness of AGCNN (17) under complex conditions, comparative analyses are conducted against five advanced baseline neural networks, namely ZNN (Tan et al., 2024), PFTZNN (Jin et al., 2023a), RTRNN (Kong et al., 2023), FAGNNZNN (Dai et al., 2022), IERNN-GVN (Yan et al., 2024) and ANLDNS (Liufu et al., 2025). The simulations are performed over a horizon of 10, s with randomly generated initial states. Since parameter tuning is critical for handling practical noise interference (Guo et al., 2025), AGCNN (17) parameters are set to $\gamma = 2$ and $\varsigma = 2.5$. To ensure a fair comparison, the parameters of the competing neural networks are set to the optimal values reported in their original works, and all simulations are conducted under identical settings.

As shown in Fig. 4 and Table 3, all neural networks, including AGCNN (17), are evaluated under four conditions: noise-free, constant noise $\boldsymbol{\eta}(t) = [e, e^2, e^3, e^4]^{\mathrm{T}}$, periodic noise $\boldsymbol{\eta}(t) = [e \sin t, 0.1 \cos 2t, \sin(et), 10 \cos(0.01t)]^{\mathrm{T}}$, and bounded random noise with each component uniformly distributed in $[-1, 1]$.

The convergence trajectories in Fig. 4 and the quantitative results in Table 3 collectively demonstrate the robustness of AGCNN (17) under different noise conditions. In the noise-free case, AGCNN (17) achieves the fastest convergence, with a CT of approximately 9.70×10^{-3} s, while maintaining high precision. Under noisy conditions, the performance gap widens significantly. While ZNN (Tan et al., 2024), PFTZNN (Jin et al., 2023a), RTRNN (Kong et al., 2023), FAGNNZNN (Dai et al., 2022),



(a) Noise-free (b) Constant noise (c) Periodic noise (d) Random noise

Fig. 4. Error convergence trajectories of different neural networks for Example 1 under different noise conditions. (a) Noise-free. (b) Constant noise. (c) Periodic noise. (d) Random noise.

Table 3
Performance comparison of AGCNN (17) and competing neural networks under different noise conditions.

Noise	Neural Networks	CT (s)	MSSRE	ASSRE
Noise-free	AGCNN (17)	$\mathbf{9.70\times10^{-3}}$	2.38×10^{-5}	3.70×10^{-6}
	ZNN (Tan et al., 2024)	4.66×10^{0}	3.41×10^{-5}	7.65×10^{-6}
	PFTZNN (Jin et al., 2023a)	2.90×10^{-1}	2.05×10^{-5}	1.93×10^{-6}
	RTRNN (Kong et al., 2023)	2.14×10^{-1}	7.49×10^{-5}	7.09×10^{-6}
	FAGNNZNN (Dai et al., 2022)	8.81×10^{-1}	4.22×10^{-4}	1.36×10^{-5}
	IERNN-GVN (Yan et al., 2024)	2.75×10^{0}	8.08×10^{-5}	6.39×10^{-6}
	ANLDNS (Liufu et al., 2025)	7.67×10^{-2}	$\mathbf{1.33\times10^{-6}}$	$\mathbf{3.42\times10^{-7}}$
Constant noise	AGCNN (17)	$\mathbf{1.11\times10^{-3}}$	3.07×10^{-4}	5.86×10^{-5}
	ZNN (Tan et al., 2024)	4.88×10^{0}	1.81×10^{0}	1.81×10^{0}
	PFTZNN (Jin et al., 2023a)	5.60×10^{-1}	2.15×10^{-1}	2.15×10^{-1}
	RTRNN (Kong et al., 2023)	4.86×10^{-1}	2.00×10^{-1}	2.00×10^{-1}
	FAGNNZNN (Dai et al., 2022)	1.58×10^{-1}	4.93×10^{-1}	4.93×10^{-1}
	IERNN-GVN (Yan et al., 2024)	5.28×10^{0}	$\mathbf{5.84\times10^{-5}}$	$\mathbf{4.34\times10^{-6}}$
	ANLDNS (Liufu et al., 2025)	7.66×10^{-2}	2.72×10^{-2}	2.72×10^{-2}
Periodic noise	AGCNN (17)	$\mathbf{1.86\times10^{-3}}$	$\mathbf{2.95\times10^{-3}}$	$\mathbf{5.07\times10^{-5}}$
	ZNN (Tan et al., 2024)	1.22×10^{0}	1.34×10^{0}	1.01×10^{0}
	PFTZNN (Jin et al., 2023a)	2.08×10^{-1}	1.87×10^{-1}	1.42×10^{-1}
	RTRNN (Kong et al., 2023)	1.67×10^{-1}	8.96×10^{-2}	6.35×10^{-2}
	FAGNNZNN (Dai et al., 2022)	8.11×10^{-2}	8.35×10^{-1}	5.98×10^{-1}
	IERNN-GVN (Yan et al., 2024)	7.07×10^{-1}	1.19×10^{-1}	8.52×10^{-2}
	ANLDNS (Liufu et al., 2025)	4.31×10^{-2}	2.41×10^{-2}	1.87×10^{-2}
Random noise	AGCNN (17)	$\mathbf{1.94\times10^{-2}}$	$\mathbf{3.38\times10^{-4}}$	$\mathbf{3.46\times10^{-5}}$
	ZNN (Tan et al., 2024)	2.49×10^{0}	5.96×10^{-1}	5.49×10^{-1}
	PFTZNN (Jin et al., 2023a)	1.23×10^{0}	6.87×10^{-2}	5.51×10^{-2}
	RTRNN (Kong et al., 2023)	1.23×10^{0}	5.06×10^{-2}	3.82×10^{-2}
	FAGNNZNN (Dai et al., 2022)	1.17×10^{0}	8.04×10^{-2}	3.79×10^{-2}
	IERNN-GVN (Yan et al., 2024)	3.15×10^{0}	8.19×10^{-3}	2.45×10^{-3}
	ANLDNS (Liufu et al., 2025)	4.55×10^{-2}	1.38×10^{-2}	8.54×10^{-3}

Table 4
Performance comparison of AGCNN (17) and competing neural networks for Example 2 under different noise conditions.

Noise	Neural Networks	CT (s)	MSSRE	ASSRE
Noise-free	AGCNN (17)	$\mathbf{1.06\times10^{-2}}$	1.47×10^{-4}	7.20×10^{-5}
	ZNN (Tan et al., 2024)	7.45×10^{0}	4.37×10^{-3}	1.68×10^{-3}
	PFTZNN (Jin et al., 2023a)	2.69×10^{-1}	4.95×10^{-4}	4.21×10^{-5}
	RTRNN (Kong et al., 2023)	2.46×10^{-1}	5.14×10^{-4}	9.80×10^{-6}
	FAGNNZNN (Dai et al., 2022)	8.80×10^{-1}	1.89×10^{-4}	9.90×10^{-6}
	IERNN-GVN (Yan et al., 2024)	2.16×10^{0}	3.82×10^{-4}	1.02×10^{-5}
	ANLDNS (Liufu et al., 2025)	7.35×10^{-2}	$\mathbf{2.25\times10^{-6}}$	$\mathbf{8.67\times10^{-7}}$
Constant noise	AGCNN (17)	$\mathbf{2.31\times10^{-3}}$	$\mathbf{1.48\times10^{-4}}$	2.24×10^{-5}
	ZNN (Tan et al., 2024)	2.41×10^{0}	1.33×10^{0}	1.33×10^{0}
	PFTZNN (Jin et al., 2023a)	5.44×10^{-1}	2.63×10^{-1}	2.63×10^{-1}
	RTRNN (Kong et al., 2023)	5.98×10^{-1}	2.45×10^{-1}	2.45×10^{-1}
	FAGNNZNN (Dai et al., 2022)	6.39×10^{-1}	2.68×10^{-1}	2.68×10^{-1}
	IERNN-GVN (Yan et al., 2024)	3.59×10^{0}	2.46×10^{-4}	$\mathbf{1.26\times10^{-5}}$
	ANLDNS (Liufu et al., 2025)	7.65×10^{-2}	3.33×10^{-2}	3.33×10^{-2}
Periodic noise	AGCNN (17)	$\mathbf{1.84\times10^{-3}}$	$\mathbf{1.25\times10^{-4}}$	$\mathbf{2.07\times10^{-5}}$
	ZNN (Tan et al., 2024)	9.57×10^{-1}	9.30×10^{-1}	7.24×10^{-1}
	PFTZNN (Jin et al., 2023a)	2.22×10^{-1}	2.10×10^{-1}	1.58×10^{-1}
	RTRNN (Kong et al., 2023)	3.65×10^{-1}	1.95×10^{-1}	1.46×10^{-1}
	FAGNNZNN (Dai et al., 2022)	7.76×10^{-2}	3.03×10^{-1}	1.90×10^{-1}
	IERNN-GVN (Yan et al., 2024)	7.43×10^{-1}	1.42×10^{-1}	8.67×10^{-2}
	ANLDNS (Liufu et al., 2025)	4.31×10^{-2}	2.70×10^{-2}	2.15×10^{-2}
Random noise	AGCNN (17)	$\mathbf{8.46\times10^{-3}}$	$\mathbf{1.60\times10^{-4}}$	$\mathbf{4.34\times10^{-5}}$
	ZNN (Tan et al., 2024)	1.75×10^{0}	4.63×10^{-1}	4.24×10^{-1}
	PFTZNN (Jin et al., 2023a)	1.26×10^{0}	8.56×10^{-2}	6.78×10^{-2}
	RTRNN (Kong et al., 2023)	1.23×10^{0}	2.15×10^{-1}	4.80×10^{-2}
	FAGNNZNN (Dai et al., 2022)	1.16×10^{0}	8.04×10^{-2}	1.21×10^{-1}
	IERNN-GVN (Yan et al., 2024)	1.60×10^{0}	2.94×10^{-4}	7.79×10^{-4}
	ANLDNS (Liufu et al., 2025)	5.50×10^{-2}	1.45×10^{-2}	1.02×10^{-2}

and IERNN-GVN (Yan et al., 2024) exhibit error spikes on the order of 10^0 or severe oscillations under constant and periodic disturbances, AGCNN (17) effectively suppresses these disturbances, with the MSSRE remaining on the order of 10^{-3} to 10^{-4}, an order of magnitude below that of ANLDNS (Liufu et al., 2025), which attains the lowest MSSRE in the noise-free case but degrades to the order of 10^{-2} once noise is present. Compared with the robust IERNN-GVN (Yan et al., 2024), AGCNN (17) still demonstrates competitive accuracy under random noise, with an MSSRE of 3.38×10^{-4}, while offering substantially higher convergence efficiency. These results further confirm the strong acceleration capability and robustness of AGCNN (17) under diverse noise conditions.

5.4. Scalability validation

To further examine the scalability of AGCNN (17), a more challenging high-dimensional TVNO problem is considered.

Example 2. Consider the following TVNO problem:

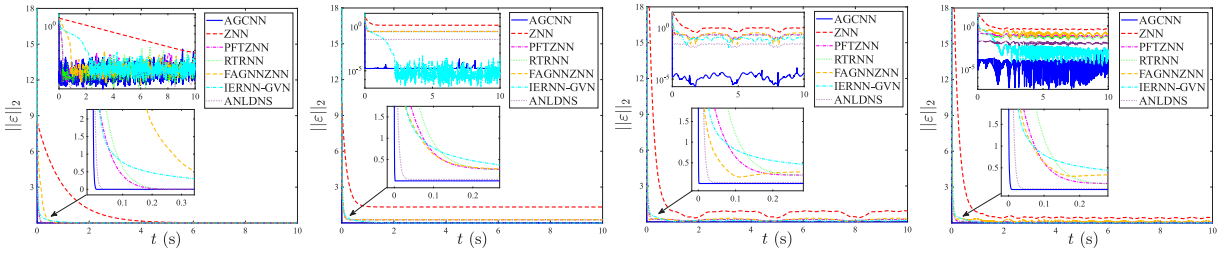
$$\min_{\boldsymbol{x}(t)\in\mathbb{R}^8} Q(\boldsymbol{x}(t),t), \tag{25}$$

where $\boldsymbol{x}(t)=[x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8]^{\mathrm{T}}$ and $Q(\boldsymbol{x}(t),t)$ is defined as

$$\begin{aligned}Q(\boldsymbol{x}(t),t) &= (x_1+t)^2+(x_2-t)^2+0.5(x_3-e^{-t})^2\\ &\quad+0.3(x_4+\sin(t))^2+(x_5+\ln(t+1))^2\\ &\quad+0.4(x_6)^2+0.5x_1x_2+x_3x_4-0.8\cos(tx_5)\\ &\quad+(x_1+\sin(2t))x_3+(x_4-\cos(t))x_2\\ &\quad+0.2x_5x_6+\sin(tx_6)\\ &\quad+0.6(x_7-\cos(2t))^2+0.5(x_8+e^{-2t})^2\\ &\quad+0.3x_6x_7+0.2x_7x_8+(x_8-\sin(t))x_7.\end{aligned}$$

In Example 2, the constant noise is defined as $\boldsymbol{\eta}(t)=[e,e^2,e^3,e^4,e^5,e^6,e^7,e^8]^{\mathrm{T}}$, and the periodic noise is defined as $\boldsymbol{\eta}(t)=[e\sin t,0.1\cos 2t,\sin et,10\cos 0.01t,2.5\sin 5t,1.5\cos 3t,2.0\sin 0.5t,0.8\cos 4t]^{\mathrm{T}}$. Furthermore, it is noteworthy that the parameters for each model in solving Example 2 are identical to those employed in solving Example 1.

The experimental results visualized in Fig. 5 and summarized in Table 4 quantitatively validate the scalability and generalizability of AGCNN (17) in solving more complex TVNO problems. In terms of convergence speed, AGCNN (17) consistently outperforms all baseline models, achieving a CT on the order of 10^{-3} s to 10^{-2} s across all conditions,



(a) Noise-free (b) Constant noise (c) Periodic noise (d) Random noise

Fig. 5. Error convergence trajectories of different neural networks for Example 2 under different noise conditions. (a) Noise-free. (b) Constant noise. (c) Periodic noise. (d) Random noise.

whereas the competing methods generally require 10^{-1} s to several seconds. In terms of robustness under high-dimensional noise, ZNN (Tan et al., 2024), PFTZNN (Jin et al., 2023a), RTRNN (Kong et al., 2023), and FAGNNZNN (Dai et al., 2022) fail to maintain high precision, with MSSRE deteriorating to the order of 10^{-1} or 10^{0} under constant and periodic disturbances, while ANLDNS (Liufu et al., 2025), which attains the lowest MSSRE in the noise-free case, degrades to the order of 10^{-2} under all three noise conditions. Notably, although the robust IERNN-GVN (Yan et al., 2024) performs adequately under constant and random noise, it still deteriorates under periodic disturbances, whereas AGCNN (17) maintains high precision with an MSSRE of 1.25×10^{-4}. These results confirm that AGCNN (17) preserves its strong acceleration capability and robustness even as the problem dimensionality and noise complexity increase.

6. Application

In this section, AGCNN (17) is employed to perform trajectory tracking with a Franka Emika Panda manipulator. The effectiveness of AGCNN (17) is validated through simulation and physical experiments.

6.1. Experimental setup

According to the forward kinematic model, the relationship between the Cartesian coordinates of the end-effector, $\boldsymbol{p}(t) \in \mathbb{R}^3$, and the joint angles, $\boldsymbol{\Theta}(t) \in \mathbb{R}^7$, is described by the nonlinear mapping $\boldsymbol{p}(t) = \mathcal{K}(\boldsymbol{\Theta}(t))$. It is worth noting that $\mathcal{K}(\cdot)$ is constructed from nominal kinematic parameters, whose deviations from the actual ones constitute a dominant source of absolute positioning error (Chen et al., 2025, 2024b; Li et al., 2022, 2021). Since such parameter errors are quasi-static, they are treated as pre-compensated in the present experiments. Due to the redundancy, this inverse kinematics problem is underdetermined. To achieve precise motion generation while managing the redundant degrees of freedom, the trajectory tracking task is formulated as a TVNO problem rather than a conventional inverse-kinematics problem.

Specifically, the tracking control equation is defined to minimize the deviation between an actual trajectory $\boldsymbol{p}_a(t)$ and a desired path $\boldsymbol{p}_d(t)$:

$$\min_{\boldsymbol{\Theta}} \frac{1}{2} \|\boldsymbol{p}_a(t) - \boldsymbol{p}_d(t)\|_2^2, \tag{26}$$

where $\boldsymbol{p}_a(t) = \mathcal{K}(\boldsymbol{\Theta}(t))$. To solve this problem, it is further transformed into finding the zero of the error function defined by the Jacobian transpose condition:

$$\sigma(\boldsymbol{\Theta}(t), t) = \boldsymbol{J}^{\mathrm{T}}(\boldsymbol{\Theta}(t))(\boldsymbol{p}_a(t) - \boldsymbol{p}_d(t)) = 0, \tag{27}$$

where $\boldsymbol{J}(\boldsymbol{\Theta}(t))$ is the Jacobian matrix. AGCNN (17) is then employed to solve (27) dynamically and generate a continuous joint trajectory $\boldsymbol{\Theta}(t)$.

The manipulator is tasked with tracking a cat-shaped closed spatial curve, which imposes stringent requirements on continuous path following. To simulate realistic engineering environments, dynamic noise $\boldsymbol{\eta}(t)$ is artificially injected in the simulation experiment. The detailed experimental configuration is summarized in Table 5.

For comparison, PFTZNN (Jin et al., 2023a), RTRNN (Kong et al., 2023), FAGNNZNN (Dai et al., 2022), and IERNN-GVN (Yan et al., 2024) are employed to solve the same trajectory tracking task under noise interference. To quantitatively evaluate the tracking accuracy and convergence performance, three metrics are employed: the Maximum Position Error (MPE), the Mean Squared Error (MSE), and the Integral Squared Error (ISE). Their mathematical definitions are as follows:

$$\begin{cases} \mathrm{MPE} = \max\limits_{t\in[0,T]} \left\|\mathcal{K}(\boldsymbol{\Theta}(t)) - \boldsymbol{p}_d(t)\right\|_2, \\ \mathrm{MSE} = \dfrac{1}{T}\displaystyle\int_0^T \left\|\mathcal{K}(\boldsymbol{\Theta}(t)) - \boldsymbol{p}_d(t)\right\|_2^2 dt, \\ \mathrm{ISE} = \displaystyle\int_0^T \left\|\mathcal{K}(\boldsymbol{\Theta}(t)) - \boldsymbol{p}_d(t)\right\|_2^2 dt, \end{cases} \tag{28}$$

where T denotes the total tracking time.

6.2. Trajectory tracking

Fig. 6 presents the experimental results of AGCNN (17) for the trajectory tracking task. Specifically, Fig. 6(a) and (b) illustrate the trajectory tracking performance in the Cartesian task space. Fig. 6(a) shows the projection of the trajectory onto the X–Y plane. It can be observed that the actual trajectory generated by the manipulator (red curve) closely matches the desired reference path (blue solid curve), effectively capturing the complex geometric features of the cat-shaped contour. Fig. 6(b) provides a three-dimensional view of the manipulator motion, showing the continuous evolution of the links as the end-effector moves along the prescribed path.

Fig. 6(c) demonstrates the kinematic feasibility of the solution. Notably, the joint velocities remain consistently within the physical saturation limits. Fig. 6(d) further evaluates the tracking accuracy of AGCNN (17). Over time, the position error of AGCNN (17) is confined within 10^{-6} m. This result confirms that AGCNN (17) can effectively handle complex trajectory constraints with excellent accuracy.

As illustrated in Fig. 7, the baseline neural networks exhibit varying degrees of failure under identical noise-injected conditions. Specif-

Table 5
Experimental configuration and parameters for the Franka Emika Panda manipulator.

Parameter	Value / Description
Manipulator	Franka Emika Panda (7-DOF)
Test Trajectory	A cat-shaped closed curve
Task Duration (T)	20 s
Sampling Frequency	1 kHz ($\tau = 0.001$ s)
Initial Joint Position $\boldsymbol{\Theta}_0$	$[0, -\frac{\pi}{4}, 0, -\frac{3\pi}{4}, 0, \frac{\pi}{2}, \frac{\pi}{4}]^{\mathrm{T}}$ rad
Initial Joint Velocity $\dot{\boldsymbol{\Theta}}_0$	$\boldsymbol{0}_{7\times1}$ rad/s
Velocity Limit $\lvert\dot{\boldsymbol{\Theta}}\rvert$	$\pi/6$ rad/s (for all joints)
Processor (Sim)	Intel Core i9-13900K / Windows 11
Processor (Physical)	Intel Core i7-4720HQ (8 GB RAM) / Ubuntu 20.04 LTS
Software	MATLAB R2022a / CoppeliaSim v4.10
Noise input $\boldsymbol{\eta}(t)$ (simulation only)	$\boldsymbol{J}^{\mathrm{T}}(\boldsymbol{\Theta})[r(t), \cos(t^2), 1]^{\mathrm{T}}$, $r(t) \in [-1, 1]$

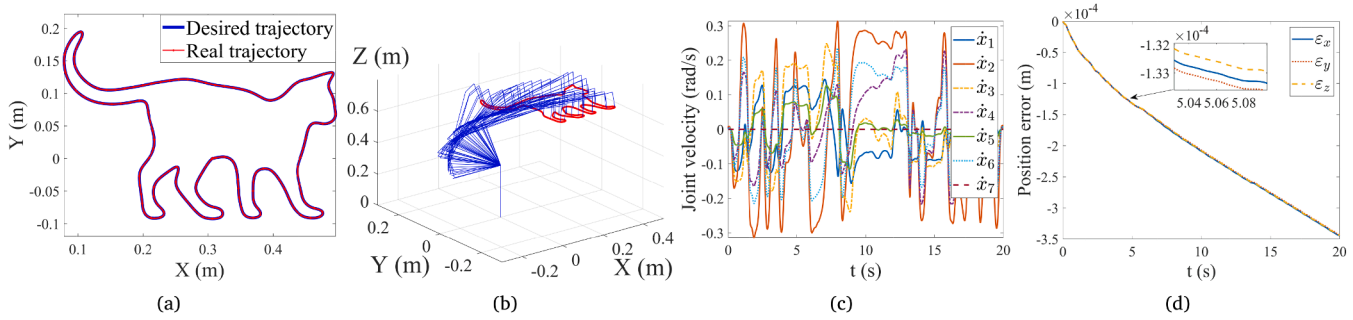


Fig. 6. Simulation results of the Franka Emika Panda manipulator tracking a cat-shaped trajectory using AGCNN (17). (a) Comparison of the desired and actual trajectories in the 2D plane. (b) 3D motion snapshots. (c) Profiles of joint angular velocities. (d) Time-varying position error components in Cartesian space.

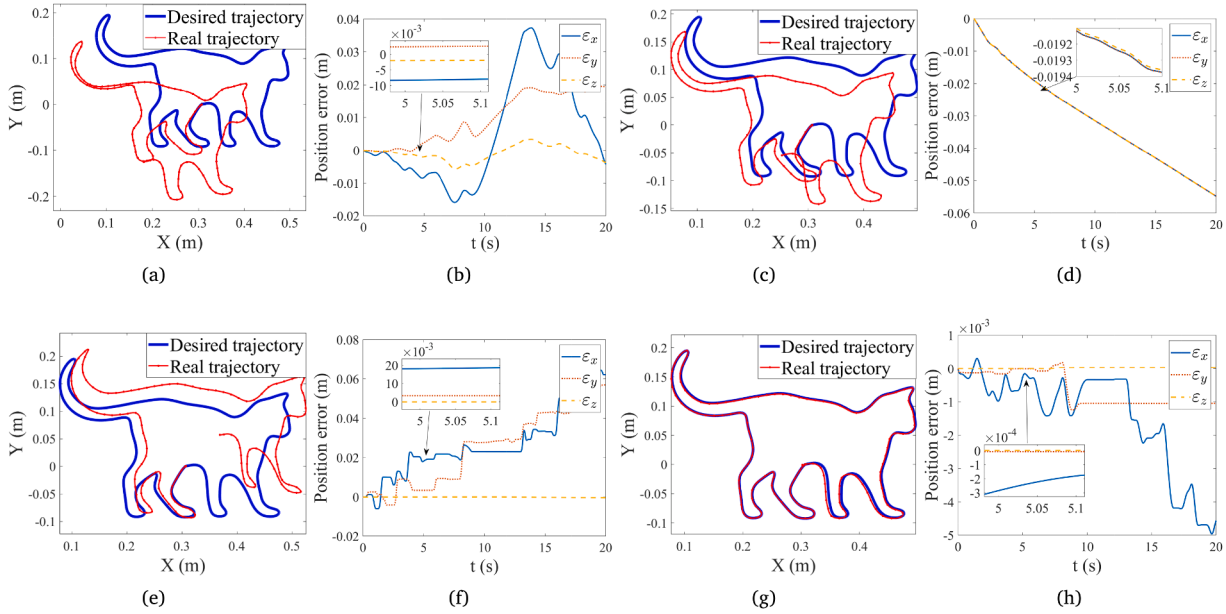


Fig. 7. Simulation results of the Franka Emika Panda manipulator tracking a cat-shaped trajectory using baseline neural networks. (a)–(b) Results of PFTZNN (Jin et al., 2023a). (c)–(d) Results of RTRNN (Kong et al., 2023). (e)–(f) Results of FAGNNZNN (Dai et al., 2022). (g)–(h) Results of IERNN-GVN (Yan et al., 2024). In each group, the left subfigure compares the desired and actual trajectories in the 2D plane, and the right subfigure shows the time-varying position error components in Cartesian space.

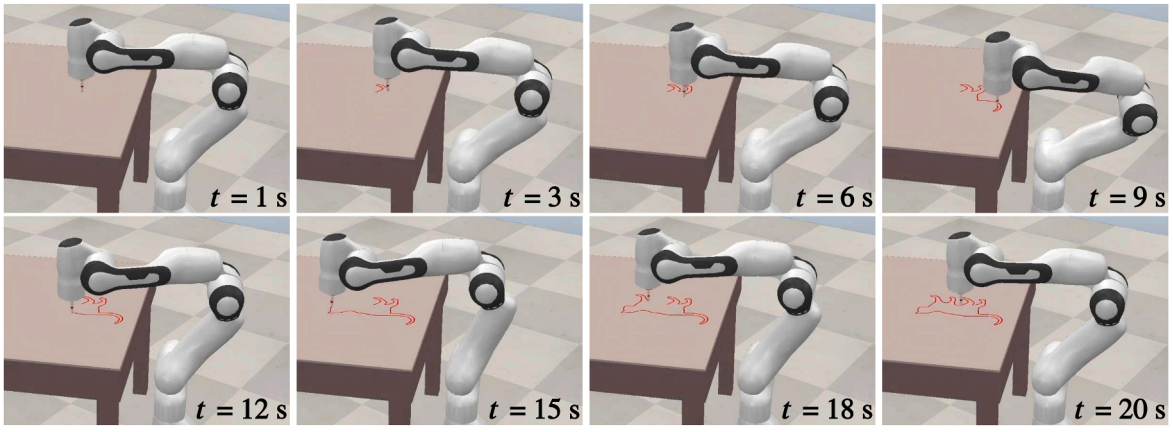


Fig. 8. Snapshots of the Franka Emika Panda manipulator performing a trajectory tracking task in CoppeliaSim.

ically, PFTZNN (Jin et al., 2023a), shown in Fig. 7(a), RTRNN (Kong et al., 2023), shown in Fig. 7(c), and FAGNNZNN (Dai et al., 2022), shown in Fig. 7(e), all struggle to handle redundancy resolution for the complex nonconvex curve. As a result, severe trajectory distortions and large deviations from the desired path appear, particularly at the sharp turns of the cat limbs and tail. The corresponding position error profiles in Fig. 7(b), (d), and (f) further reveal numerical instability, characterized by large drifts and non-convergent behavior. Although IERNN-GVN (Yan et al., 2024), shown in Fig. 7(g), can trace the general outline, it still suffers from noticeable chattering and a residual error on the order of 10^{-3} m, as shown in Fig. 7(h).

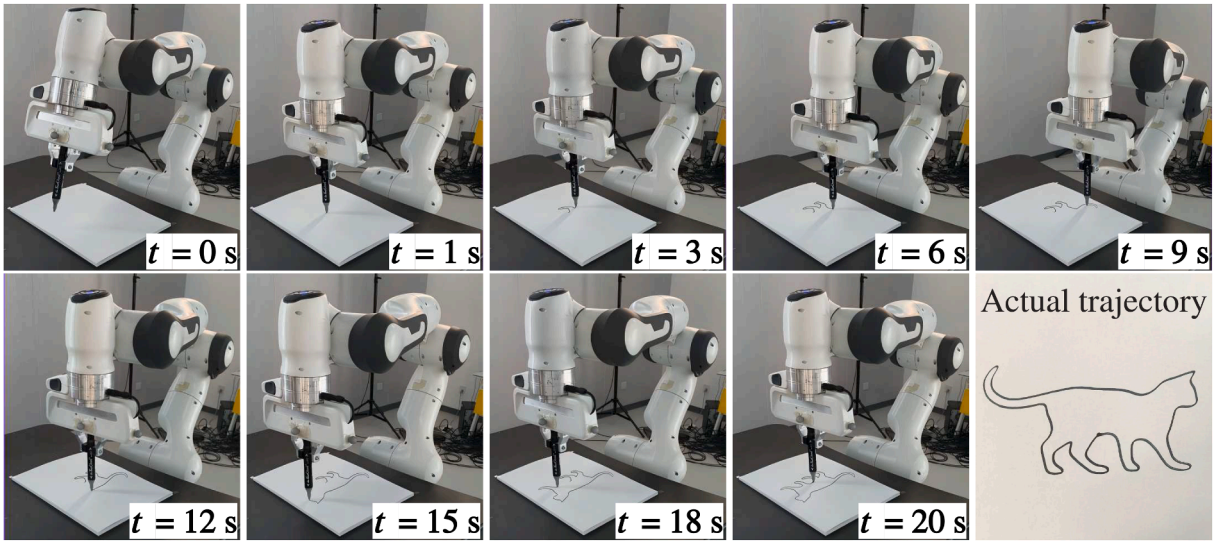


Fig. 9. Snapshots of the Franka Emika Panda manipulator performing a trajectory tracking task in a real physical environment.

Table 6
Performance comparison of AGCNN (17) and four baseline models for the Franka Emika Panda manipulator under noisy conditions.

Neural Networks	CT (s)	MPE	MSE	ISE
AGCNN (17)	**2.29**	$\mathbf{5.96\times10^{-4}}$	$\mathbf{1.44\times10^{-7}}$	$\mathbf{2.89\times10^{-6}}$
PFTZNN (Jin et al., 2023a)	5.41	2.18×10^{-1}	1.74×10^{-2}	3.48×10^{-1}
RTRNN (Kong et al., 2023)	5.11	9.48×10^{-2}	3.50×10^{-3}	6.90×10^{-2}
FAGNNZNN (Dai et al., 2022)	5.07	8.60×10^{-2}	2.00×10^{-3}	3.94×10^{-2}
IERNN-GVN (Yan et al., 2024)	3.02	5.10×10^{-3}	4.92×10^{-6}	9.84×10^{-5}

Table 6 provides a quantitative comparison between AGCNN (17) and the baseline neural networks. In terms of computational efficiency, AGCNN (17) achieves the fastest convergence with a CT of 2.29 s, outperforming all baseline models. In terms of tracking accuracy, AGCNN (17) also shows a substantial advantage. Its MPE is 5.96×10^{-4}, whereas the best competing method, IERNN-GVN (Yan et al., 2024), remains on the order of 10^{-3}. Similar trends can be observed in the MSE and ISE metrics, where AGCNN (17) consistently yields values several orders of magnitude lower than those of the competing methods. These results further verify the effectiveness and robustness of AGCNN (17) in practical trajectory tracking tasks.

6.3. Physical validation

To further validate the trajectory-tracking performance of AGCNN (17) in practical scenarios, experiments are conducted on a Franka Emika Panda manipulator in both CoppeliaSim and a real-world physical environment. In both settings, the manipulator is controlled at 1 kHz to track the same cat-shaped reference trajectory, and a pen is attached to the end-effector to visually record the executed path. The disturbances in this setting are inherent to the platform rather than artificially injected. Since the task is formulated at the kinematic level, AGCNN (17) outputs a joint-velocity command executed by the internal controller, and joint friction, encoder quantization, and unmodeled dynamics are represented collectively by the discrepancy $\delta(t)$ between commanded and realized velocities, entering the optimality-error dynamics as $\eta(t) = D(t)\delta(t)$. Under the velocity limit $\pi/6$ rad/s, $\delta(t)$ is bounded, so $\|\eta(t)\|_2 \leq \varpi$ and Theorem 4 applies through this aggregate bound rather than through a model of any individual effect.

As shown in Fig. 8, the simulated manipulator follows the prescribed trajectory smoothly and progressively over time, and the generated path gradually forms the complete contour of the target curve. Fig. 9 further presents the corresponding results obtained in the physical test bed. Despite the presence of real-world uncertainties, including joint friction, encoder quantization, and unmodeled dynamics, the end-effector still traces the desired curve with high consistency with the simulation results. The snapshots at different time instants show that the physical robot completes the cat-shaped trajectory in a continuous and stable manner, without visible oscillations, abrupt deviations, or emergency stops. These results confirm that the proposed method is not only effective in simulation but also practically applicable to real-time trajectory tracking tasks in physical robotic systems.

7. Conclusion

In this paper, AGCNN is proposed for solving TVNO problems. Different from the existing acceleration approaches, AGCNN explicitly addresses two structural sources of the lagging error in GNNs: the feedforward velocity compensation mechanism cancels the drift induced by time variation, while the adaptive spectral compensation mechanism maintains effective feedback correction under weak-curvature conditions. Together with EPMAF, these mechanisms are integrated to reduce the lagging error while achieving fast and robust convergence. Theoretically, the two compensation mechanisms are proved to jointly eliminate the lagging error of GNNs, and the global finite-time convergence and robustness of AGCNN under bounded disturbances are established, together with explicit bounds on the convergence time and steady-state error. Numerical simulations demonstrate that AGCNN achieves faster convergence, higher accuracy, and stronger robustness than several representative neural networks under different noise conditions. In addition, experiments on a Franka Emika Panda manipulator further verify the effectiveness of AGCNN in trajectory-tracking tasks. Future work will focus on extending the proposed framework to more complex time-varying optimization problems, such as those involving inequality constraints, large-scale settings, and broader robotic applications.

CRediT authorship contribution statement

Shangfeng Du: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Duo Liu:** Writing – review & editing, Writing – original draft, Visualization, Validation; **Yang Si:** Methodology, Funding acquisition, Formal analysis; **Haoen Huang:** Writing – review & editing, Software, Methodology; **Zihao Zhang:** Writing – review & editing, Visualization, Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the Graduate Supervisor Team under Grant No. 0225005401117, the Fundamental Research Funds for the Central Universities under Grant Nos. 2024CDJGF-003 and 2024CDJGF-019, and the National Natural Science Foundation of China under Grant No. 62402070.

References

Andrychowicz, M., Denil, M., Gómez, S., Hoffman, M. W., Pfau, D., Schaul, T., Shillingford, B., & de Freitas, N. (2016). Learning to learn by gradient descent by gradient descent. In *Adv. Neural Inf. Process. Syst.* (*vol. 29*).

Boyd, S., & Vandenberghe, L. (2004). Convex optimization. Cambridge university press.

Chen, J., Pan, Y., & Zhang, Y. (2024a). ZNN continuous model and discrete algorithm for temporally variant optimization with nonlinear equation constraints via novel TD formula. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *54*(7), 3994–4004. https://doi.org/10.1109/TSMC.2024.3374754

Chen, T., Yang, W., Li, S., & Luo, X. (2025). Data-driven calibration of industrial robots: A comprehensive survey. *IEEE/CAA Journal of Automatica Sinica*, *12*(8), 1544–1567.

Chen, T., Yang, W., Zhang, Z., & Luo, X. (2024b). An efficient industrial robot calibrator with multiplaner constraints. *IEEE Transactions on Industrial Informatics*, *20*(12), 14341–14350.

Dai, J., Chen, Y., Xiao, L., Jia, L., & He, Y. (2022). Design and analysis of a hybrid GNN-ZNN model with a fuzzy adaptive factor for matrix inversion. *IEEE Transactions on Industrial Informatics*, *18*(4), 2434–2442.

Datta, R., Pradhan, S., & Bhattacharya, B. (2016). Analysis and design optimization of a robotic gripper using multiobjective genetic algorithm. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *46*(1), 16–26. https://doi.org/10.1109/TSMC.2015.2437847

Fu, D., Huang, H., Wei, L., Xiao, X., Jin, L., Liao, S., Fan, J., & Xie, Z. (2022). Modified Newton integration algorithm with noise tolerance applied to robotics. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *52*(4), 2134–2144. https://doi.org/10.1109/TSMC.2021.3049386

Fu, Z., Zhang, Y., & Tan, N. (2023). Gradient-feedback zhang neural network for unconstrained time-variant convex optimization and robot manipulator application. *IEEE Transactions on Industrial Informatics*, *19*(10), 10489–10500.

Goldfarb, D. (1980). Curvilinear path steplength algorithms for minimization which use directions of negative curvature. *Mathematical Programming*, *18*(1), 31–40.

Guo, D., Zhang, C., Cang, N., Jia, Z., Xue, S., Zhang, W., Li, S., & Wang, Y. (2025). Harmonic noise rejection zeroing neural network for time-dependent equality-constrained quadratic program and its application to robot arms. *IEEE Transactions on Industrial Informatics*, *21*(2), 1279–1288.

Guo, D., & Zhang, Y. (2012). Novel recurrent neural network for time-varying problems solving [research frontier]. *IEEE Computational Intelligence Magazine*, *7*(4), 61–65.

Huang, H., Zeng, Z., & Wang, J. (2025). A trust-region projection neural network for nonlinear programming. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(10), 18737–18749. https://doi.org/10.1109/TNNLS.2025.3576600

Jiang, C., Xiao, X., Liu, D., Huang, H., Xiao, H., & Lu, H. (2021). Nonconvex and bound constraint zeroing neural network for solving time-varying complex-valued quadratic programming problem. *IEEE Transactions on Industrial Informatics*, *17*(10), 6864–6874.

Jin, J., Chen, W., Chen, C., Chen, L., Tang, Z., Chen, L., Wu, L., & Zhu, C. (2023a). A predefined fixed-time convergence ZNN and its applications to time-varying quadratic programming solving and dual-arm manipulator cooperative trajectory tracking. *IEEE Transactions on Industrial Informatics*, *19*(8), 8691–8702.

Jin, J., Zhu, J., Zhao, L., Chen, L., Chen, L., & Gong, J. (2023b). A robust predefined-time convergence zeroing neural network for dynamic matrix inversion. *IEEE Transactions on Cybernetics*, *53*(6), 3887–3900.

Kong, Y., Wu, J., Jiang, Y., & Wu, H. (2023). Robust terminal recurrent neural network for finding exact solution of the TVQP problem with various noises. *IEEE Transactions on Industrial Informatics*, *19*(5), 6907–6916.

Li, S., Chen, S., & Liu, B. (2012). Accelerating a recurrent neural network to finite-time convergence for solving time-varying sylvester equation by using a sign-bi-power activation function. *Neural Processing Letters*, *37*(2), 189–205.

Li, W., Wu, H., & Jin, L. (2024). A lower dimension zeroing neural network for time-variant quadratic programming applied to robot pose control. *IEEE Transactions on Industrial Informatics*, *20*(10), 11835–11843.

Li, Z., Li, S., Bamasag, O. O., Alhothali, A., & Luo, X. (2022). Diversified regularization enhanced training for effective manipulator calibration. *IEEE Transactions on Neural Networks and Learning Systems*, *34*(11), 8778–8790.

Li, Z., Li, S., & Luo, X. (2021). An overview of calibration technology of industrial robots. *IEEE/CAA Journal of Automatica Sinica*, *8*(1), 23–36.

Liao, S., Liu, J., Qi, Y., Huang, H., Zheng, R., & Xiao, X. (2022a). An adaptive gradient neural network to solve dynamic linear matrix equations. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *52*(9), 5913–5924.

Liao, S., Liu, J., Qi, Y., Huang, H., Zheng, R., & Xiao, X. (2022b). An adaptive gradient neural network to solve dynamic linear matrix equations. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *52*(9), 5913–5924.

Liao, Z., Gong, W., Yan, X., Wang, L., & Hu, C. (2020). Solving nonlinear equations system with dynamic repulsion-based evolutionary algorithms. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *50*(4), 1590–1601. https://doi.org/10.1109/TSMC.2018.2852798

Liufu, Y., Jin, L., & Li, S. (2025). Adaptive noise-learning differential neural solution for time-dependent equality-constrained quadratic optimization. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(9), 17253–17264.

Liufu, Y., Lv, X., Jin, L., & Li, S. (2026). Orthogonal projected gradient differential neural solution to linear and quadratic constrained optimization: Theory and applications. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, (pp. 1–13). https://doi.org/10.1109/TSMC.2026.3664485

Luo, J., Li, J., Xiao, L., & Li, J. (2025). A novel ZNN-based chaos synchronization controller and its application in secure voice communications. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *55*(11), 7890–7899. https://doi.org/10.1109/TSMC.2025.3599651

Lyu, J., Zhang, H., Gao, G., Ding, Y., Sun, F., & Zhang, J. (2025). Dynamic grasping based on reinforcement learning and differentiable MPC. *IEEE Transactions on Automation Science and Engineering*, *22*, 23822–23834. https://doi.org/10.1109/TASE.2025.3629383

Tan, J., Shang, M., & Jin, L. (2024). Metaheuristic-based RNN for manipulability optimization of redundant manipulators. *IEEE Transactions on Industrial Informatics*, *20*(4), 6489–6498. https://doi.org/10.1109/TII.2023.3348830

Tang, J., Su, Z., Liu, J., & Jin, L. (2025). Robust model predictive control of position and orientation of redundant manipulators. *IEEE Transactions on Automation Science and Engineering*, *22*, 16201–16212. https://doi.org/10.1109/TASE.2025.3572441

Xiao, L., Zhang, Y., Huang, W., Jia, L., & Gao, X. (2024). A dynamic parameter noise-tolerant zeroing neural network for time-varying quaternion matrix equation with applications. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(6), 8205–8214.

Xie, Z., Liu, M., Su, Z., Sun, Z., & Jin, L. (2025). A data-driven obstacle avoidance scheme for redundant robots with unknown structures. *IEEE Transactions on Industrial Informatics*, *21*(2), 1793–1802.

Yan, J., Jin, L., luo, X., & Li, S. (2024). Modified RNN for solving comprehensive sylvester equation with TDOA application. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(9), 12553–12563.

Yan, J., Su, Z., Ma, X., & Jin, L. (2025). An obstacle avoidance scheme for manipulators aided by noise-tolerant neural dynamics. *IEEE Transactions on Industrial Informatics*, *21*(1), 703–712.

Yang, Y., Li, S., Wu, P., & Feng, W. (2026). Design and analysis of an inverse-free zeroing neural network model for matrix decomposition. *IEEE Transactions on Emerging Topics in Computational Intelligence*, *10*(3), 2516–2528. https://doi.org/10.1109/TETCI.2026.3668587

Yang, Y., Li, S., Wu, P., Moshayedi, A. J., & Feng, W. (2025a). Design and analysis of a novel zeroing neural network for matrix decomposition under data loss and noise interference. *Applied Mathematical Modelling*, *147*, 116201.

Yang, Y., Wu, P., Katsikis, V. N., Li, S., & Feng, W. (2025b). A novel real-time noise-resilient zeroing neural network and its applications to matrix problem solving. *Mathematics and Computers in Simulation*, *240*, 1083–1099. https://doi.org/10.1016/j.matcom.2025.01.006

Yang, Y., Wu, P., Pham, D. T., Li, S., & Feng, W. (2025c). A novel zeroing neural network with activation function-enhanced convergence for efficient data matrix decomposition. *Neurocomputing*, *647*, 130495.

Zhang, F., Su, Z., Xie, Z., & Jin, L. (2024). Discrete-time noise-resilient neural dynamics for model predictive motion-force control of redundant manipulators. *IEEE Computational Intelligence Magazine*, *20*(11), 13101–13112.

Zhang, Y., Chen, K., & Tan, H.-Z. (2009). Performance analysis of gradient neural network exploited for online time-varying matrix inversion. *IEEE Transactions on Automatic Control*, *54*(8), 1940–1945.

Zhang, Y., & Ge, S. S. (2005). Design and analysis of a general recurrent neural network model for time-varying matrix inversion. *IEEE Transactions on Neural Networks*, *16*(6), 1477–1490.

Zhang, Y., Jiang, D., & Wang, J. (2002). A recurrent neural network for solving sylvester equation with time-varying coefficients. *IEEE Transactions on Neural Networks*, *13*(5), 1053–1063.

Zhang, Y., Jin, L., & Ke, Z. (2012). Superior performance of using hyperbolic sine activation functions in ZNN illustrated via time-varying matrix square roots finding. *Computer Science and Information Systems*, *9*(4), 1603–1625.

Zhang, Y., Li, Z., Guo, D., Chen, K., & Chen, P. (2013). Superior robustness of using power-sigmoid activation functions in z-type models for time-varying problems solving. In *Proc. 2013 Int. Conf. Mach. Learn. Cybern.* (pp. 759–764). IEEE.

Zhang, Y., Zhang, J., & Weng, J. (2023). Dynamic moore–penrose inversion with unknown derivatives: Gradient neural network approach. *IEEE Transactions on Neural Networks and Learning Systems*, *34*(12), 10919–10929. https://doi.org/10.1109/TNNLS.2022.3171715

Zhao, B., Shi, G., & Liu, D. (2023). Event-triggered local control for nonlinear interconnected systems through particle swarm optimization-based adaptive dynamic programming. *IEEE Transactions on Systems, Man, and Cybernetics, Systems*, *53*(12), 7342–7353. https://doi.org/10.1109/TSMC.2023.3298065